

IPL Match Predictor

Project Report

Project Title:	IPL Match Predictor
Technology:	Java
Project Type:	Desktop GUI Application
Reg.no:	25BAI10801

1 1. INTRODUCTION

IPL Match Predictor is a standalone Java desktop application that allows a user to select two franchises from the Indian Premier League (IPL) and receive an instant, easy-to-read prediction of the likely winner along with a win-probability breakdown for each team. The application is built using Java Swing, requires no external dependencies or internet connection, and is intended as both a fun utility for cricket fans and a compact example of desktop GUI development in Java.

2 2. PROBLEM STATEMENT

Cricket fans frequently want a quick sense of which of two IPL teams is favoured to win a match, but doing this manually requires comparing form, squad strength, and history across several sources.

There is no lightweight, offline tool that gives an instant, transparent win-probability estimate for two chosen teams without requiring an account, an internet connection, or a complex analytics dashboard. This project solves that gap with a simple, self-contained desktop tool.

3 3. FUNCTIONAL REQUIREMENTS

- The system shall allow the user to select a team from a dropdown list for Team 1.
- The system shall allow the user to select a team from a dropdown list for Team 2.
- The system shall prevent the user from selecting the same team for both Team 1 and Team 2, and shall show a warning dialog if attempted.
- The system shall calculate and display a predicted winner when the user clicks the “Predict Winner” button.
- The system shall display the win probability (as a percentage) for each selected team.
- The system shall display a logo image for each selected team if an image file is available.
- The system shall display a fallback badge with the team's initials if no logo image file is found.
- The system shall run an initial prediction automatically when the application starts, using the default team selections.

4 4. NON-FUNCTIONAL REQUIREMENTS

- Usability: The interface must be simple enough for a first-time user to operate without instructions.
- Performance: Prediction results must be computed and displayed instantly (no perceptible delay) since the calculation is a lightweight arithmetic operation.
- Portability: The application must run on any OS with a standard JDK (Windows, macOS, Linux) without modification.
- Reliability: The application must not crash if logo image files are missing; it must degrade gracefully to the initials badge.
- Maintainability: Team data (names, logos, strength ratings) is centralized in one method (setupData) to make it easy to add or update teams.
- No External Dependencies: The application must run using only the standard Java library, with no third-party libraries or network access required.

5 5. SYSTEM ARCHITECTURE

The application follows a single-class, event-driven desktop architecture typical of small Java Swing programs. There is no client-server or database layer; all data and logic live in memory within one running process.

5.1 LAYERS

- Data Layer - In-memory LinkedHashMap objects (teamLogos, teamStrength) holding static team data, populated once at startup in `setupData()`.
- UI Layer - Swing components (JFrame, JPanel, JComboBox, JButton, JLabel) built in `setupUI()` and helper methods (`buildHeader`, `buildBody`, `buildResultCard`) that assemble the header, team selectors, logo badges, and result card.
- Logic / Controller Layer - Event handlers (`onPredict`, `updateLogos`) triggered by Swing `ActionListeners`, which read the current dropdown selections, compute probabilities, and update the UI labels directly.

5.2 DATA FLOW

User selects teams in the dropdowns -> `ActionListener` fires -> `updateLogos()` refreshes the badge/logo panels -> user clicks Predict Winner -> `onPredict()` reads both selections, looks up strength values, computes percentages, determines the winner -> result and probability `JLabels` are updated directly on the Event Dispatch Thread. No background threads or asynchronous calls are needed given the trivial computation cost.

6 6. DESIGN DECISIONS & RATIONALE

- Single-file Swing application - chosen for simplicity and zero-dependency distribution; appropriate given the small scope of the tool.
- LinkedHashMap for team data - preserves insertion order so the dropdown lists display teams in a predictable, consistent order.
- Strength-ratio probability formula ($\text{team strength} / \text{total strength}$) - a simple, transparent, and easily explainable model rather than a black-box or ML-based approach, keeping the tool auditable and easy to extend.
- Graceful logo fallback (initials badge) - ensures the UI never breaks or shows a broken-image icon if asset files are not bundled or found, improving robustness.
- Dark theme with a single accent color - gives the app a modern, focused look without requiring an external theming library.
- Fixed, non-resizable window - simplifies layout logic for a small, form-like utility where responsive resizing is not necessary.

7 7. IMPLEMENTATION DETAILS

The application is implemented in a single class, `IPLPredictor`, extending `JFrame`. Key implementation points:

- Team data (8 franchises) is stored as name -> logo-path and name -> strength-score pairs.

- The prediction formula computes $\text{prob1} = \text{round}(t1 / (t1+t2) * 1000) / 10.0$ and similarly for prob2, giving a one-decimal-place percentage; the team with the higher raw strength score is reported as the predicted winner.
- Input validation in `onPredict()` compares the two selected team names and shows a `JOptionPane` warning dialog if they match, returning early without updating the result.
- Logo loading (`loadLogo`) checks for a file's existence before attempting to load it, scaling any found image to 64x64 pixels; if the file does not exist, `styleBadge()` falls back to an abbreviation generated from the team name's initials.
- The UI is composed with `BorderLayout` for the main frame and `BoxLayout/GridLayout` for sub-panels, using `EmptyBorder` and `CompoundBorder` for consistent spacing and a card-like appearance.
- `main()` sets the cross-platform Look and Feel and starts the UI on the Swing Event Dispatch Thread via `SwingUtilities.invokeLater`, following standard Swing best practice.

8 8. SCREENSHOTS / RESULTS

Insert screenshots of the running application here before final submission, for example:

- Main window on launch, showing the default team matchup and prediction.
- A predicted result after selecting two different teams (e.g. Chennai Super Kings vs Mumbai Indians).
- The warning dialog shown when the same team is selected on both sides.

Screenshot placeholder - replace this section with actual application screenshots captured after running IPLPredictor.java.

9 9. TESTING APPROACH

As a small UI utility, testing was performed manually rather than with an automated test suite. The following checks were used to validate correctness:

- Launch test - confirm the application starts without exceptions and shows an initial default prediction.
- Duplicate-selection test - select the same team on both sides and confirm the warning dialog appears and no result is overwritten.
- Prediction correctness test - verify that for known strength values, the team with the higher score is always reported as the winner, and the two probabilities sum to approximately 100%.
- Missing-asset test - run the app with no logos/ folder present and confirm every team renders its initials badge instead of crashing or showing a broken image icon.
- Full combination sweep - cycle through all valid team pairings to confirm no combination produces an error or an out-of-range probability.

For future iterations, unit tests could be added (e.g. with `JUnit`) around a refactored, UI-independent prediction function to allow automated regression testing.

10 10. CHALLENGES FACED

- Asset availability - team logo images are not bundled with the source, so the UI needed a robust fallback path (initials badge) to avoid a poor experience when assets are missing.

- Keeping the UI responsive with plain Swing - without a layout framework, careful use of nested BorderLayout/BoxLayout/GridLayout panels and borders was required to achieve a clean, card-like look.
- Balancing simplicity vs. realism in the prediction model - using a static strength score keeps the tool simple and dependency-free, at the cost of not reflecting real-time form, injuries, or venue conditions; this trade-off was accepted given the project's scope.
- Cross-platform consistency - ensuring fonts, colors, and the cross-platform Look and Feel render consistently across Windows, macOS, and Linux without relying on OS-specific styling.